AZINE

Architecture

Front end, back end, hosting and apps, and why each piece is the way it is

OCTOBER 29, 2026
AZINE.IO · MAIN @ 9EB9BE6

1. What azine is
2. The system at a glance
3. Hosting and deployment
4. Two ways in: the website and the home-screen app
5. Back end: the server
6. Content: catalogs, sources and enrichment
7. Finding things: filters, groups and prompt search
8. Front end: the four letters
9. Operations: limits, costs, backups
10. Decision log
11. Keeping this document current

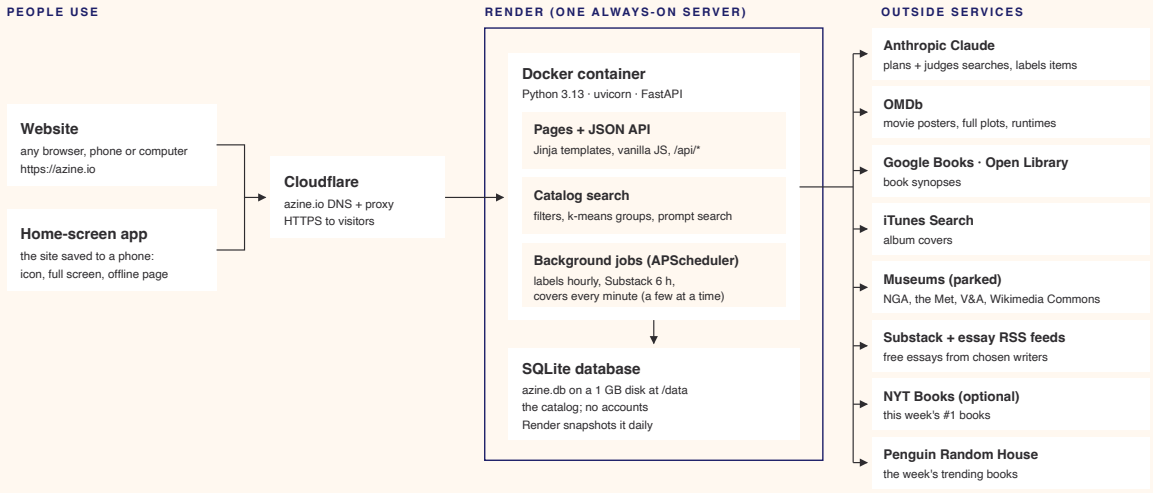
01 What azine is

azine is a catalog of things worth your time: essays, movies, albums and books (E-M-A-B), the same for everyone. Every entry is a real work from a curated catalog, mostly pre-2021 (plus festival films from 2022 on) and nothing in it is AI-written. There are no accounts, no personalized picks and no voting. You find things three ways: by category, by when the work was made (and, for movies, albums and books, by country), and by saying what you're after ("I want to read about x").

WHY IT'S SHAPED THIS WAY

The product promise is "the best of the internet without the filler." Until Oct 27, 2026 azine delivered it as a personal board built from a taste answer and votes; it is now a catalog anyone can browse, with no sign-up standing between a visitor and the work. Claude is used only behind the scenes: to understand a search prompt, to judge which works fit it, and to label catalog items. It never writes what you read.

02 The system at a glance



Everything runs as one Python server on Render, with its SQLite database on an attached disk. Cloudflare serves the azine.io name; outside services are called only to understand search prompts, label items, and fetch covers and synopses.

What happens when someone searches

1. Cloudflare resolves `azine.io` and passes the request to the Render service over HTTPS.
2. Browsing by category, date or group is a plain database query (`GET /api/catalog`): no Claude, no account.
3. A prompt (`POST /api/search`) is first checked in plain code for prompt-injection phrasing ("ignore all previous instructions"); if it has any, the reader is told so and nothing goes to Claude. Otherwise it goes to Claude Sonnet once to become a search plan, inside the filters already set.
4. The server lines up every work inside the filters, most similar first, then Claude Haiku scores each for how well it fits the whole prompt; everything scoring 75+ comes back, best first, however many that is (~25 s for 1,700 films).
5. Anything shown without a cover is queued, and a background job fetches it within a minute or so.

03 Hosting and deployment

PIECE	SETUP
Host	Render web service, Docker runtime, region Oregon, plan 0.5 CPU / 512 MB (~$7/month), always on
Image	`Dockerfile`: `uv` + Python 3.13 slim; dependencies in their own layer; runs `uvicorn azine.main:app --proxy-headers` on `$PORT`
Disk	1 GB persistent disk mounted at `/data`; `AZINE_DB_PATH=/data/azine.db`. It keeps the catalog's fetched covers, synopses and labels across deploys.
Config	`render.yaml` (Blueprint): disk, health check `/health`, env vars. Secrets (`ANTHROPIC_API_KEY`, `OMDB_API_KEY`, `GOOGLE_BOOKS_API_KEY`, `NYT_API_KEY`) are set in the dashboard, never in git.

Domain	`azine.io` registered on Cloudflare: `A @ → 216.24.57.1`, `CNAME www → azine.onrender.com`. Render issues the certificate; `azine.onrender.com` also still works.
Deploys	Every push to `main` builds and deploys. With a disk attached Render stops the old server first, so there's ~1 minute of downtime. Migrations run on start.
Security	No accounts, so no cookies or passwords. uvicorn trusts the proxy's `X-Forwarded-*` headers, which the search rate limit uses to tell visitors apart. Search prompts pass a plain-code prompt-injection check before reaching Claude.

WHY ONE SERVER, SQLITE AND A DISK

azine's needs are modest: one database file, a few background jobs, and a few hundred requests a day at most. A single always-on process with SQLite on a persistent disk is the simplest setup that does all of that: no separate database server, no job queue, no cluster. The constraints this brings are deliberate: exactly one instance (the SQLite file and the in-process scheduler both assume it), and the disk must exist (the server now refuses to start without it, with a message saying so). The way to grow is a bigger instance; moving to Postgres and a separate worker only becomes necessary if azine ever needs more than one server.

WHY RENDER (AND NOT THE FREE TIER, FLY.IO, OR VERCEL)

- **Vercel, Netlify, Cloudflare Pages, Heroku** were ruled out: no persistent disk and/or no always-on process, so they'd require Postgres plus an external scheduler.
- **Fly.io** was set up first (it fits technically), then swapped for **Render** because Render is driven from a web dashboard rather than a command-line tool, and auto-deploys from GitHub.
- **Render's free tier** sleeps after 15 idle minutes (which would stop the hourly refresh) and can't have a disk.
- **Docker** keeps it portable: the same image runs on any host that offers a disk at `/data` and HTTPS.

WHY 512 MB IS ENOUGH

Measured peak was ~310 MB with the old board (re-measure for the catalog: each category's embeddings and groups are now held in memory). It used to be ~630 MB: the National Gallery of Art fetch held two ~40 MB CSV slices as text (GitHub gzips them, so an "8 MB" byte range is ~40 MB decoded). It now streams the rows and keeps only the three fields it uses. Memory grows with *simultaneous* activity, not total accounts. Upgrade to 1 CPU / 2 GB if the logs ever show it running out.

04 Two ways in: the website and the home-screen app

Website

Server-rendered pages at azine.io; works in any modern browser, phone or desktop.

Home-screen app

The same website, saved to a phone's home screen from the browser ("Add to Home Screen"). It gets an icon, opens full screen and shows an offline page with no connection. Built from `manifest.webmanifest`, icons and a service worker (`/sw.js`); nothing to publish.

There is no iOS app.

Caching and offline

The service worker is **network-first for everything**: pages and the API are never served from a cache (the board is personal and changes daily), and `/static/` files are cached only as an offline fallback. With no connection, a page load shows `static/offline.html`. The stylesheet's address carries its file's modification time (`style.css?v=…`, from `static_url()`), so every edit is a new URL and browsers can never pair a new page with an old stylesheet.

WHY NO APP STORE APP

The home-screen app already gives phones an icon and a full-screen azine, and every deploy updates it at once. A store app adds a review for each release, and Apple turns down apps that only wrap a website (guideline 4.2) unless they add something native.

WHY NETWORK-FIRST CACHING

Originally static files were served cache-first. After the board redesign, a browser paired the new `board.html` with its cached old `style.css` and showed an unstyled page. Versioned stylesheet URLs plus a network-first service worker make that impossible while keeping the offline page.

05 Back end: the server

Code layout (`src/azine/`)

MODULE	JOB
main.py	Builds the FastAPI app: static files, routers, manifest/service worker routes, `/health`; on start runs migrations, seeds the catalog, starts the scheduler
db.py · models.py	SQLAlchemy engine/session (SQLite), the catalog and museum-art tables
migrations/	Alembic migrations, applied automatically on every start
routers/	`catalog.py` (the API), `pages.py` (pages, robots, sitemap); `art.py` is parked, not mounted

catalog/	`filters.py` (category + date, sorting, acclaim), `embeddings.py`, `groups.py` (k-means groups), `search.py` (prompt search), `enrich.py` (covers and synopses), `details.py` (fetching them in bulk into `data/details/`)
claude.py	The one structured-output request to Claude, shared by search and labeling
content_sources/ · data/	Catalog loaders and live sources (Substack, RSS, museums, OMDb, iTunes, books); curated catalogs and vocabularies
labeling.py · catalog_builder.py	Subject/tone labels (hand-edited JSON + Claude for new items); offline tool that builds the big catalogs from source lists
scheduler.py · seed.py · art_refresh.py	Background jobs; catalog upserts on start; museum image rotation (parked)

HTTP endpoints

ROUTE	PURPOSE
GET /	The front page (`templates/index.html` + `static/js/front.js`): the four letters, then the catalog, all through the API below
GET /about	The owner's letter (public; text in `data/about_letter.txt`)
GET /api/catalog	The catalog filtered by `category` (any of the four, repeatable), `year_from`/`year_to`, `country` (repeatable; essays have none) and optionally a `group`; sorted by acclaim, newest, oldest or a seeded shuffle; paged
GET /api/catalog/groups	Each category's groups, sized to the date and country filters
GET /api/catalog/meta · /{id}	Counts, year ranges and countries (with counts) per category, for building the filter controls; one item
POST /api/search	A prompt plus the same filters: every work Claude judges a 75%+ fit, best first, with the plan it read from the prompt. 30 searches per visitor per hour; a prompt with prompt-injection phrasing is refused (400) before Claude sees it
/robots.txt · /sitemap.xml · /favicon.ico	For search engines: keep out of /api; list the public pages
/health · /manifest.webmanifest · /sw.js	Health check for Render; the installable-app files

Data model

TABLE	HOLDS
content_items	The catalog: category, title, creator, year, source, url, image, synopsis, tags, subjects, tones, countries, tier, runtime, publish date
art_images	Museum images (parked until the new front end wants them)

Migration 0006 dropped every table that held a reader's data (users, taste signals, recommendations and votes, saved items, palettes, outlet connections). The code that used them is at the git tag `personal-recommender-final`.

WHY MIGRATIONS RUN ON START

The database must never be deleted to pick up a schema change. Alembic migrations are applied automatically when the server starts, locally and on Render, so a deploy that changes the schema upgrades the live database in place.

Background jobs (APScheduler, inside the server process)

JOB	EVERY	DOES
label_new_items	1 hour	Claude labels up to 200 catalog items that have no subjects/tones yet, and up to 600 movies, albums and books with no countries
refresh_substack	6 hours	Pulls new free posts from the chosen Substack newsletters
refresh_trending_books	1 week	Adds Penguin Random House's ten trending books of the week that aren't already in the catalog (children's books left out). Also runs a few minutes after each start, since deploys restart the server more often than weekly
fetch_wanted_images	1 min	Fetches covers/synopses for up to 15 items readers were just shown without one
backfill_images	30 min	Fetches 20 more, most acclaimed first, inside the free API quotas

06 Content: catalogs, sources and enrichment

CATEGORY	CATALOG (≈ ITEMS)	WHERE IT COMES FROM
Movies	1,900	Cross-referenced "greatest films" lists, 1940–2021, plus 2022+ films that screened at Cannes, Sundance, Toronto or Hong Kong. Trailers for some; Library of Congress Screening Room switched off.
Books	3,500 + weekly	Novels from cross-referenced "greatest novels" lists, 1810 onward; every book that reached #1 on the New York Times fiction or nonfiction list since 1976 (~1,580, from Wikipedia's yearly lists, built by `azine-build-catalog bestsellers` into `books_bestsellers_list.py`); Penguin Random House's trending books each week; and this week's NYT #1s with a key
Albums	1,800	Rolling Stone's 500 Greatest Albums, plus BBC Music Magazine's 31 greatest classical albums (`albums_classical_list.py`), plus ~1,300 from Latin America, South Korea and Africa: winners and nominees of the Latin Grammys and Grammys (album of the year, urban/música urbana, Latin pop, Latin rock, global music), the Korean Music Awards (a critics' jury, each genre's album prize), MAMA and Golden Disc album of the year, the Headies, Rolling Stone Argentina's top 10, and OkayAfrica readers' 100 best African albums. Built by `azine-build-catalog albums` into `albums_extended_list.py`; the Rolling Stone 500 still sorts first by acclaim. Each album is tiered "instrumental" (no words) or "lyrical".

Essays	2,000 + live	About 1,500 picked by people, not algorithms: Longform.org's recommendations (2010–2022; its essay and memoir tags, every "Best Article" and its classics) and the National Magazine Award winners for essays, columns, criticism and feature writing (1977–2025, from Wikipedia); and 60 well-known essays The Atlantic ran before 1931 (Thoreau, Emerson, Douglass, Du Bois, Muir, Zitkala-Ša), now public domain and linked to their free full text on Wikisource or Project Gutenberg (`essays_atlantic_archive.py`, hand-kept). Built by `azine-build-catalog essays` into `essays_extended_list.py`; `list_count` drives the acclaim sort. Plus a hand-edited author list, non-paywalled RSS feeds, and chosen Substack newsletters (free posts only).
Art	live	National Gallery of Art open data, the Met (incl. costume), V&A, Wikimedia Commons museum paintings; Library of Congress paused

Enrichment: fetched in bulk, saved with the code

Covers, synopses, links and runtimes for the whole catalog are fetched on a developer's machine with `uv run azine-fetch-details` (`catalog/details.py`) and saved in `data/details/movie.json`, `album.json` and `book.json`, one item per line. They are committed like the labels, and seeding applies them on every start, so the live site has them as soon as it's deployed. The command looks up only what's still missing, saves as it goes, and stops after 40 misses in a row (a quota or a block), so it can be rerun the next day. Anything still missing is fetched by the server the old way: an item *shown* without a cover is queued and fetched within a minute or so, and a slower job backfills the rest, most acclaimed first. Either way, `catalog/enrich.py` does the lookup and stores it on the row (pages never wait on these lookups):

- **Movies:** OMDb poster, IMDb link, runtime and the *full* plot (a paragraph, not OMDb's one-liner). A record counts only if its director matches the catalog's (OMDb's title lookup returns namesakes: a podcast episode for Se7en); otherwise OMDb's search is tried, including with "and" spelled "&".
- **Albums:** cover art and the album's Apple Music page from iTunes Search (one lookup gives both). Only a result by the same artist counts (or one with exactly the same, longer title), so an album iTunes doesn't have gets no cover rather than someone else's.
- **Books:** synopsis from Google Books (publisher descriptions, several sentences; needs a free key), else Open Library. No cover: books, like essays, show their words (owner's call), so a book is looked up only while its synopsis is short. Descriptions are cleaned (HTML, markdown, "From Amazon.com", ALL-CAPS banners), cut at a sentence end past ~900 characters, and non-English text is skipped.

A fuller synopsis replaces a catalog one-liner and survives restarts; each item is looked up at most once per server run.

WHY IN BULK, AND WHY SAVED IN THE REPOSITORY

The trickle (20 every 30 minutes, plus what readers are shown) left most of the live catalog bare: search results came back as plain cards with no synopsis, and what was fetched on one machine never reached the other. Fetching once, ahead of time, and committing the result gives every copy of azine the same covers. Free API quotas are small (OMDb: 1,000 requests/day; Google Books: per-key daily quota; iTunes: ~20 a minute), so the command paces itself and a run may take more than a day.

Labels: subjects and tones

Every item is labeled on two axes from one shared vocabulary (`data/labels_vocab.py`): **subjects** (what it's about) and **tones** (how it feels). Curated labels live in `data/catalog_labels.json` (hand-editable, applied on every start); new items are labeled by Claude hourly. Because the vocabulary is shared, one search reads across categories: "melancholy" means the same on a film and an album. Labels also decide which items can be grouped (below): an item known only by its title has nothing to be grouped by.

Countries (movies, albums, books; not essays): a movie's production countries, an album's artist's, a book's author's (an émigré gets both), main one first, at most three, as today's common English names ("Russia" for Soviet work, "Germany" for East or West). Labeled by Claude from what it knows of the work and creator, kept in the same JSON file under `"countries"`.

07 Finding things: filters, groups and prompt search

1. Category, date and country (no Claude)

Any of essays, movies, albums and books, a range of years the work was made (essays use the year published; a full date orders Substack posts within it), and any of the countries. Works with no known year drop out once a date range is set; essays drop out once a country is. Sorted by acclaim (how many source lists name it, scaled within its category so a list count and an album rank compare), newest, oldest or a seeded shuffle that stays put while you page.

2. Embeddings

Each item is a TF-IDF vector blending its labels (genres, subjects, tones; 70% of the weight) with its title and synopsis text (30%). Labels carry most of the signal: measured on the real catalog, 91–94% of same-genre pairs land closer than the nearest 10% of random pairs, versus ~50% for text alone.

3. Groups (no Claude)

`catalog/groups.py` clusters each category with k-means over those embeddings (about one group per 50 items, 6–40 per category) and names each group from the labels its members share far more than the rest of the category ("eerie · mental health · dark"). Only items with a genre tag or a label are grouped: clustered anyway, the title-only items all fell into one meaningless group a third of the catalog wide. They join a group once labeled. Groups are rebuilt when the catalog or its labels change, so group ids are for use within a visit.

4. Prompt search

1. **Check** (no Claude). `catalog/prompt_guard.py` looks for the English wordings of prompt injection (ignore / disregard / forget … instructions, "system prompt", new instructions, chat-template markers), including spaced-out, digit-for-letter and look-alike-letter spellings. A match stops the search with a 400 and a message saying why; only the command forms count, so "a soldier who ignores his orders" still searches.
2. **Plan** (one Sonnet call). Claude reads the prompt, never the catalog, and returns the 1–4 distinct things it asks for ("facets"), each with its own categories, years and countries (only ever narrowing the reader's filters: "funny 90s albums" is albums from 1990–99, "Korean films" is movies from South Korea), subjects

and tones from the label vocabulary, keywords, well-known works that typify it, and the groups that fit. "The sea, and also funny 90s albums" is two facets; averaging them would find work that is neither.

3 **Order** (local). Each facet becomes one point in each category's embedding space, and every work inside the filters is lined up nearest first, works Claude named at the front. Facets share the line by how central each is to the prompt, taking turns: the multi-exemplar idea from Pinterest's PinnerSage, with Claude doing the splitting. This only sets the order and what falls past the 2,000 cap.

4 **Judge** (Haiku, batches of 60, 32 at once). Claude scores every work in the line 0–100: it first works out the prompt's essential parts, and a work fits only if all of them hold. Works scoring 63–74 get a second look in a fresh batch and the average of the two. Everything at 75+ comes back, best first, however many that is.

WHY A JUDGE, AND WHY A 75% BAR INSTEAD OF A TOP 10

Owner's rule: a search returns the works that really fit, not a fixed number. For "a girly film about finding yourself", *Roman Holiday* fits and *Pride and Prejudice* doesn't: the mood matches, but it isn't about self-discovery. Similarity in a TF-IDF space can tell "girly" and "a young woman's story", not that distinction, so it only gathers candidates; Claude, which knows the works, makes the call. Judging only the nearest few hundred missed it: labeled romantic and royal, *Roman Holiday* ranks ~350th by similarity for that prompt, so every work inside the filters is judged. Haiku makes that affordable (1,700 films is a few cents) and, on this prompt, agreed with the owner where Sonnet didn't (Sonnet read *Roman Holiday* as a romance, 62); the plan, one short call that decides which works get named, uses Sonnet, whose plans were far better. Scores near the line vary by a few points between runs (78–85 for *Roman Holiday*), hence the second look. A search's plan is cached, so the same prompt and filters cost one plan.

WHY TF-IDF

TF-IDF (scikit-learn) separates genres well at a catalog of thousands; a neural embedding model would add a multi-gigabyte dependency for little gain at this size. Worth revisiting if the catalog reaches tens of thousands.

08 Front end: the four letters

The old pages (the landing note, sign-up, the board, the quiz, saved items, the weekly page) went with accounts on Oct 27, 2026; they are at the git tag `personal-recommender-final`. The new front page (Oct 28, 2026) is one server-rendered template, `templates/index.html`, plus `static/js/front.js`, which builds everything after the first screen from the catalog API (section 05).

1 **The letters.** E, M, A, B stacked down the left in Helvetica on `#FFF8F0`, and "better for your brain" small and grey in the bottom right.

2 **The title card.** Clicking a letter grows it from where it sits into the big first letter of its title card (**E**SSAYS) while the others fade; the rest of the word and a line about the category (with the catalog's real year range) fade in beside it. The card holds for 3 seconds (a click skips it), then the page becomes that catalog. The address keeps what's open (`azine.io/#movie`, `#movie+album`), so reload and the back button work; "azine" at the top of the filter column (or Esc) goes back to a clean `azine.io`.

3. **The filter column**, on the left: the four letters again (pick one or several), a vertical timeline with the earliest year at the top (drag either end; the years show as `yyyy - yyyy`), and "Vibes", a box for the prompt search (Enter sends it).
4. **The catalog**, in 5 columns, most acclaimed first, 60 at a time as you scroll (`GET /api/catalog`). Movies and albums show only their covers; hovering shows the title, who made it, the year and the synopsis. Essays and books show their words: title, author, year, source, synopsis. A cover not yet fetched is a plain card with the title on it.
5. **While a search runs** (`POST /api/search`, ~15–25 s), the middle shows one work of art at a time from `data/landing_wall.py` (28 public-domain works from the Met, stored with the site), with its title, artist, year, a sentence of description and the museum, a new random one every 6 seconds. Then every result, in the same 5 columns. Changing a filter afterwards goes back to the catalog; Enter searches again.

Styles: section 9 of `style.css` and the FRONT PAGE block of its `:root` settings (colors, the letters' font, size and spacing, the zoom time, the title card's sizes and how long it holds, the tagline, the filter column's width, the number of columns). On a phone the letters shrink and the catalog drops to 2 columns.

WHY NO FRAMEWORK

Plain templates and CSS variables keep every visual decision in one readable file, with nothing to compile and no dependency churn. The page has four states (letters, catalog, waiting, results); a few hundred lines of plain JavaScript cover them.

WHY ART WHILE SEARCHING

A search takes Claude 15–25 seconds to judge every work in the filters. A spinner makes that feel like waiting; a painting with a line about it makes it part of the visit, and uses images already stored with the site, so it never waits on a museum.

Handwriting font

The board's fixed labels are in the owner's handwriting, as a real font: "Azine Hand" (`static/fonts/azine-hand.woff2`, 23 KB). The owner wrote the alphabet (capitals and lowercase, twice each), punctuation and a heart in a drawing app, and the labels with the digits on a second page. Both were exported as *vector* PDFs, where each pen stroke is a filled outline. `scripts/make_handwriting_font.py` groups the strokes into rows and characters, matches them to the expected letters, sets every letter exactly on the baseline (letters that hang below it, like g and the comma, are placed by the row's fitted line), evens the capitals and digits to one height (`EVEN_HEIGHTS`), merges each character's strokes (skia-pathops) and writes the font (fontTools). The two versions of each letter alternate through an OpenType `calt` rule, so double letters don't repeat exactly. Any element with `class="hand"` uses it: section headings, the details' labels, the date (month written out), the menu, the sort labels and buttons, and "Fetching your recs". Titles, synopses and values stay in the regular fonts. Settings: the HANDWRITING block in `style.css`.

WHY A FONT, NOT IMAGES OF WORDS

A font handles text that changes (the date) and any future label, stays real text (resizable, recolorable by the CSS settings, readable by screen readers), and is one small file. Cut-out word images would match the page exactly but need re-drawing for every wording change. The vector PDFs made the font lossless: the outlines are the pen strokes themselves, not a trace of pixels.

09 Operations: limits, costs, backups

TOPIC	TODAY
Cost	Render web service ~$7/month + small disk; azine.io yearly at Cloudflare; Claude API usage per search (a Sonnet plan plus Haiku judging every work in the filters: roughly 1–5 cents) and per label batch
Memory	~310 MB peak of 512 MB. Watch Render → Metrics; upgrade to 1 CPU / 2 GB if it nears ~450 MB or restarts for memory
API quotas	OMDb 1,000/day; Google Books per key. The cover jobs stay inside them; searches are capped at 30 per visitor per hour
Backups	Render snapshots the disk daily and keeps them ≥7 days (restoring loses later changes). Own copy: `scp -s` over Render SSH (see DEPLOY.md)
Logs	Render → Logs. Startup refuses to run without the `/data` disk and says so
Deploy time	Build a few minutes; a cold start on an empty database takes ~1 minute (seeding + fetching Substack)
Scaling path	1) bigger instance; 2) only if more than one server is needed: Postgres + a separate worker for the scheduled jobs

10 Decision log

Newest first. Earlier entries (before Sept 30, 2026) predate this document and are reconstructed from the code and its history.

WHEN	DECISION	WHY
Oct 29	**Every NYT #1 since 1976, and Penguin Random House's trending books weekly**	Owner's request: more books, nonfiction too. Wikipedia keeps each year's fiction and nonfiction #1s (1,578 books over 50 years); the NYT's own API only goes back to 2008. Only #1s, not every listed title, so searches stay quick. PRH's weekly page lists its ten trending books in plain HTML, so a weekly job adds the new ones.
Oct 29	**Novels become Books: N is now B**	Owner's call: the catalog takes nonfiction too, starting with New York Times #1 bestsellers. The category key, letter and labels move from "novel" to "book"; migration 0008 renames existing rows.
Oct 29	**No iOS app; its code deleted**	Owner's call: azine won't be a store app. The home-screen app already gives phones an icon and full screen, and App Store review turns down apps that only wrap a website. The Capacitor shell in `ios-app/` was removed.
Oct 29	**These documents restyled to be minimal**	Owner's request: easier to read. Fewer lines and boxes, larger type, more space.
Oct 29	**Search prompts checked for prompt injection before Claude sees them**	Anyone can type into Vibes. `prompt_guard.py` is plain code, so it can't be talked round: it refuses the common English forms ("ignore all previous instructions", "reveal your system prompt"), disguised spellings included, and tells the reader why. Refusals count toward the hourly limit. Backstop: Claude only returns a plan and scores for works already in the catalog. Considered: a small model as screener (itself injectable, and a cost per search).
Oct 28	**Public-domain Atlantic essays**	The Atlantic and The New Yorker meter every article, so the free route is work from before 1931. Sixty well-known Atlantic essays, each linked to a full free copy on Wikisource or Project Gutenberg. The New Yorker's public-domain years are few and mostly humor.
Oct 28	**Covers and synopses fetched in bulk and committed; movies checked against the director**	The slow trickle left many search results bare. `azine-fetch-details` now fetches for the whole catalog into `data/details/`, applied on every start. OMDb sometimes returned a namesake (a podcast for Se7en), so a movie's record counts only if the director matches. Novels and essays show their words, not covers (owner's call).
Oct 28	**Country labels and a country filter**	Owner's request. Claude labeled movies, albums and novels in one pass (a few dollars) instead of matching each to Wikidata. Today's countries; essays have none.
Oct 28	**Albums from Latin America, South Korea and Africa**	Owner's request: albums were almost all Anglo-American. With no good all-time critics' lists, ~1,280 come from juried awards (winners and nominees). Cover lookups now require the same artist.
Oct 28	**Front page: E M A N down the left; a click grows the letter into a title card**	Owner's Canva design. The card shows for 3 seconds before the catalog. Helvetica, to match the design.

Oct 28	**First new front page: letters in the corners, zoom into one**	Owner's design, tried first as a clickable mockup. Movies and albums show covers; essays and novels, their words. Met art fills the wait while a search runs.
Oct 28	**Essays from Longform and the National Magazine Awards**	Essays were the thinnest category, with no acclaim signal. Both sources are chosen by editors or juries, which keeps the catalog AI-free. Only Longform's essay-like picks, so essays don't swamp the rest.
Oct 27	**Search judges every work in the filters; Sonnet plans, Haiku judges**	Roman Holiday never came back for "a girly film about finding yourself": it ranked too low by similarity to be judged. Now every work is judged (~25 s, a few cents) and the bar is 75 (owner's call). Haiku planned poorly, so Sonnet plans; Haiku judges at ~1/15 the cost.
Oct 27	**azine becomes a catalog: no accounts, personal picks or votes**	Owner's direction: AI-free E-M-A-N anyone can browse. Filters and groups need no Claude; a prompt is planned and judged by Claude. Migration 0006 drops reader data; the old code is at tag `personal-recommender-final`. Considered: no Claude for search (misses "sad" vs "melancholy"); sending Claude the whole catalog (too many tokens).
Oct 9	**A third vote: "neutral"**	"Not this one" without saying anything about the kind of thing.
Oct 9	**Taste answer: 20 words minimum (was 50)**	Less to ask of a first-time visitor.
Oct 8	**Albums open in Apple Music or Spotify**	A page can't tell which app is installed, so it offers both.
Oct 8	**Landing note fits a phone; its arrow is drawn live**	The writing fills the width, and the arrow always reaches the box.
Oct 8	**Handwriting font: letters set on the line**	Each letter now sits on the baseline at one height.
Oct 2	**Landing opens on Met art; About page; SEO basics**	Art first, the same for everyone. The About page and descriptions give search engines readable text.
Oct 1	**Board labels in the owner's handwriting, as a font**	Fits azine's "made by a person" point, and stays real text.
Oct 1	**Vibes page removed from the board**	Fun, but not needed.
Oct 1	**Classical albums: BBC Music Magazine's 31**	Rolling Stone's list had none.
Oct 1	**Essay search runs in parallel with a 12-second budget**	The first sign-up stalled on a minutes-long search.

Oct 1	**Landing page is the handwritten note; Claude reads the answer during sign-up**	Hides Claude's time behind the sign-up form.
Oct 1	**Four paintings as the loading bar**	Clear stages read better than a spinner.
Oct 1	**Domain azine.io on Cloudflare**	azine.com is taken and parked for sale.
Oct 1	**Refuse to start without the /data disk**	Running without it would lose the database on every deploy.
Oct 1	**Versioned stylesheet URL + network-first service worker**	A cached old stylesheet broke the redesigned page.
Oct 1	**Movies use OMDb's full plot**	Longer synopses read better.
Oct 1	**Novel synopses and covers from Google Books / Open Library**	90% of novels had none. Claude-written synopses ruled out (no AI-written content).
Oct 1	**Showcases: synopsis right, title left, no borders**	Owner's design direction.
Oct 1	**Wall stays discrete; the rest scrolls normally**	Discrete screens made content vanish on scroll.
Oct 1	**Stream the NGA art CSVs**	Peak memory 630 → 310 MB, inside Render's 512 MB.
Oct 1	**Host on Render (replacing Fly.io)**	A dashboard, and deploys from GitHub. Free tier, Vercel, Netlify and Heroku ruled out (no disk or no always-on process).
Oct 1	**App icon is the "A"**	A square mark reads better as an icon.
Sept 30	**Installable web app (and an iOS wrapper, since shelved)**	One codebase, and updates without an App Store release.
Sept 30	**Docker image + configurable database path**	Portable; the database lives on a disk that survives deploys.
Sept 30	**Saved items and daily vibes (migration 0003)**	Saved copies outlive rotating boxes.
Sept 29	**Shared subject/tone labels + taste profile**	A taste read in one category carries into the others.

Sept 28	**SQLite + Alembic migrations applied on start**	Never delete the database to change the schema.
Earlier	**Free-text taste prompt read by Claude**	One natural answer replaces a long quiz.
Earlier	**PinnerSage-style taste clusters over TF-IDF**	Keeps distinct tastes instead of averaging them.
Earlier	**Enrich items when picked, not in bulk**	Stays inside free API quotas.
Earlier	**Server-rendered pages, vanilla JS, CSS variables**	Hand-tunable, nothing to build.

11 Keeping this document current

The sources live in `docs/architecture/` (`architecture.html`, `summary.html`, `system.svg`, `print.css`); the PDFs are `docs/azine-architecture.pdf` and `docs/azine-summary.pdf`. After any significant change (hosting, data model, a new feature or flow, a major design change, a new outside service) update the affected sections, add a row to the decision log, rebuild with `node docs/architecture/build.js`, and commit the PDFs alongside the change. `CLAUDE.md` asks Claude to do this and to share the updated PDFs.